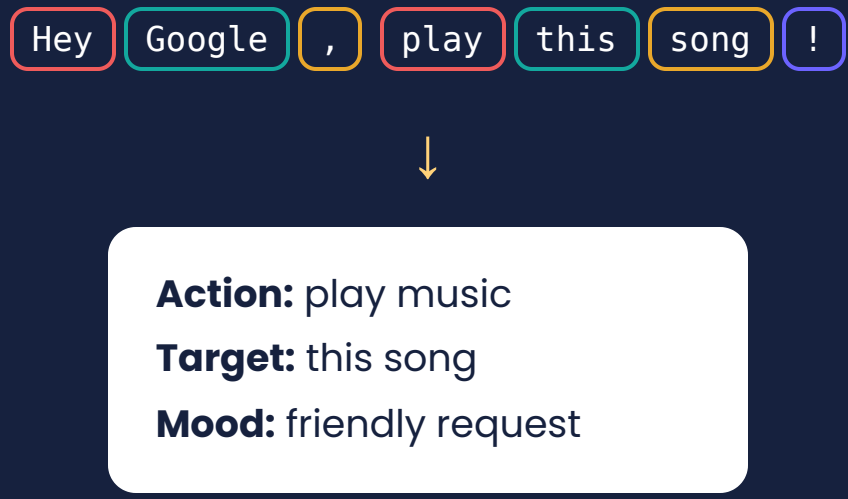
INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Natural Language Processing (NLP)

How computers learn to read, understand, and work with human language.



WARM-UP

Have you talked to a computer today?

You probably used language technology many times today without noticing it.

🎤

Voice assistants

You say “Hey Google, play this song,” and it understands you.

⌨️

Autocorrect

Your phone fixes “teh” into “the” and guesses your next word.

🌐

Translation

Apps turn a Filipino sentence into English in one second.

✉️

Spam filters

Your email hides “WIN A FREE PHONE!!!” messages for you.

💬

AI chatbots

Chat assistants read your question and write a reply.

⭐

Review ratings

Shopping apps sort reviews into happy and unhappy ones.

All of these use **Natural Language Processing**.

LEARNING GOALS

What we will learn today

By the end of this lesson, you will be able to do these six things.

1. Explain what NLP is and why it is important.
2. Describe how a computer breaks text into small pieces called tokens.
3. Identify the job of each word using parts of speech.
4. Tell the difference between stemming and lemmatization.
5. Explain what stop words are and when to remove them.
6. Show how computers find phrases using patterns.

THE BIG IDEA

What is Natural Language Processing?

Natural Language Processing (NLP) is a part of computer science and artificial intelligence that helps computers **read, understand, and use human language**.

Natural

This means the languages people speak every day, like English, Filipino, or Spanish.

Language

This means words, sentences, and the meaning behind them.

Processing

This means the steps a computer follows to analyze large amounts of text.

In short, NLP is the bridge between how **people** talk and how **computers** think.

MOTIVATION

Computers love numbers, but what about words?

✔️ Numbers are easy

Student	Height (cm)	Score
Ana	152	89
Ben	160	93
Carla	149	78

A computer can instantly find the average, the highest score, or the tallest student.

✘ Text is messy

> *“omg the food was sooo good!! but the service was kinda slow tbh... 3 stars i guess”*

A person understands this right away. A computer only sees a long string of letters with no clear structure.

NLP gives computers special techniques to turn messy text into **organized data**.

MOTIVATION

Why is human language hard for computers?

One word, many meanings

“I saw a **bat**.” Is it the animal or the baseball bat? We need context to know.

Many words, one meaning

Happy, **glad**, and **joyful** mean almost the same thing, but they look completely different.

Slang, typos, and short forms

People write “gr8,” “sooo good,” and “idk.” A computer must still understand them.

Many languages and mixes

People mix languages, like Taglish: “Sobrang ganda ng movie, super worth it!”

MOTIVATION

Why should you learn NLP?



Text is everywhere

Messages, posts, news, reviews, and school papers are all text. Much of the world's information is written in words.



It powers modern AI

AI chatbots, translators, and voice assistants are all built on the ideas you will learn today.



It solves real problems

NLP helps detect scam messages, understand customer complaints, and translate for people who need help.



It opens future careers

Machine learning engineers, data scientists, and app developers use NLP in their daily work.

REAL WORLD

NLP in action

In each example, text goes in and a useful answer comes out.

Spam detection	*“WIN a FREE phone!!! Click this link now!”*	→	**SPAM**
Sentiment analysis	*“The movie was amazing. I loved every scene!”*	→	**POSITIVE 😊**
Customer feedback	*500 written feedback forms from a store*	→	**Top complaint: slow delivery**
Voice commands	*“Hey Google, play this song.”*	→	**Action: play music**

THE BIG IDEA

NLP turns messy text into organized data

NLP uses many small steps to give text a structure that a computer can work with.

> *"The kids were running happily."*

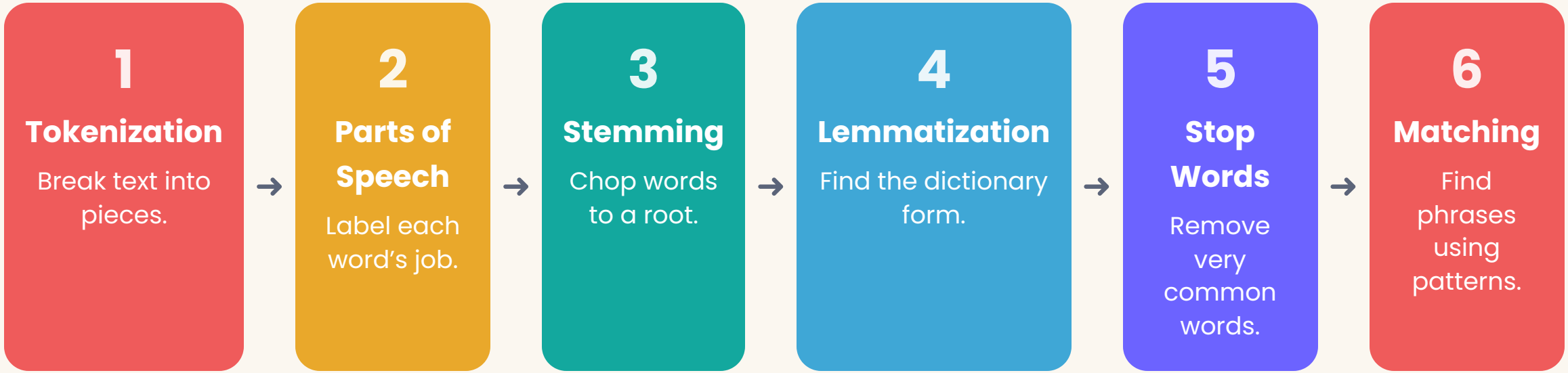


Token	Word type	Base form	Important?
The	Determiner	the	No
kids	Noun	kid	Yes
were	Helping verb	be	No
running	Verb	run	Yes
happily	Adverb	happily	Yes
.	Punctuation	.	No

ROADMAP

The NLP pipeline

NLP works like an assembly line. Each station does one job, then passes the text to the next station.



This is also our roadmap for the rest of the lesson.

OUR TOOLS

Meet spaCy and NLTK

A **library** is a ready-made toolbox of code. We do not need to build everything from zero.

spaCy

A modern and fast Python library. One function, `nlp()`, runs many steps at once: it splits, tags, and describes the text.

Best for: real projects and apps.

NLTK

The Natural Language Toolkit. It is an older library that is great for learning. It includes classic tools like the Porter and Snowball stemmers.

Best for: learning and experiments.

SIMULATED DEMO

How spaCy processes text

SIMULATED DEMO

```
import spacy

# Load the small English language model
nlp = spacy.load("en_core_web_sm")

# Give raw text to the pipeline
doc = nlp("I love learning about AI!")

for token in doc:
    print(token.text, token.pos_)
```

OUTPUT

```
I        PRON
love     VERB
learning VERB
about    ADP
AI       PROPN
!        PUNCT
```

Step 1. We load a language model. It already knows a lot about English.

Step 2. We pass our sentence into `nlp()`. This creates a pipeline object called a **Doc**.

Step 3. The Doc already contains every token and its word type. One line did all the work.

01 Tokenization

Breaking text into small, useful pieces.

TOKENIZATION

What is tokenization?

Tokenization is the process of breaking up the original text into small pieces called **tokens**.

Think of a chocolate bar

You break a big bar into small squares so it is easy to share. Tokenization breaks a sentence into small pieces so it is easy to analyze.

Example

"I love NLP!"

↓

`I` `love` `NLP` `!`

Tokens are exact pieces of the original text. Nothing is changed or shortened yet.

TOKENIZATION

Why not just split at the spaces?

Your first idea might be to split the sentence at every space. Let us test it.

“We’re moving to L.A.!”

↓ split on spaces

`“We’re` `moving` `to` `L.A.!”`

Problem 1

The quotation marks are stuck to the words. **“We’re** is treated as one strange word.

Problem 2

L.A.!” mixes a place name, an exclamation point, and a quote mark together.

We need smarter rules to split text correctly.

SIMULATED DEMO

Watch the tokenizer work, step by step

“We’re moving to L.A.!”	**Original text**
“We’re · moving · to · L.A.!”	**Split on spaces**
“ · We’re · moving · to · L.A.!”	**Prefix**
“ · We · ’re · moving · to · L.A.!”	**Exception**
“ · We · ’re · moving · to · L.A.! · ”	**Suffix**
“ · We · ’re · moving · to · L.A. · ! · ”	**Exception**
“ · We · ’re · moving · to · L.A. · ! · ”	**Done**

Highlighted pieces show what changed in each step. The tokenizer keeps going until no more rules apply.

TOKENIZATION

The four rules of the tokenizer

Prefix

Characters at the **beginning** of a word that should be split off.

`$` `(` `“`

Suffix

Characters at the **end** of a word that should be split off.

`km` `)` `,` `.` `!` `”`

Infix

Characters **in between** parts of a word that can separate it.

`-` `--` `/` `…`

Exception

A **special rule** that splits a word in a special way, or keeps it together.

`let’s → let + ’s`

`U.S. stays U.S.`

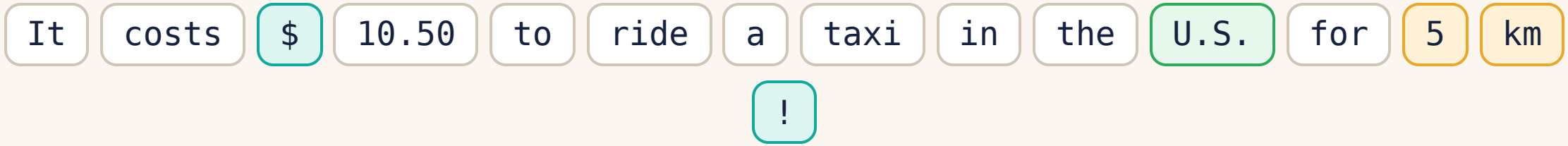
SIMULATED DEMO

Tokenizing a tricky sentence

SIMULATED DEMO

```
doc = nlp("It costs $10.50 to ride a taxi in the U.S. for 5km!")
print([token.text for token in doc])
```

OUTPUT: 15 TOKENS



- **$** and **!** were split off as a prefix and a suffix.
- **U.S.** stayed together because of an exception rule.
- **5km** became **5** and **km**, a number and a unit.

SIMULATED DEMO

Tokens have useful attributes

Every token carries extra information. The computer can check these facts instantly.

Token	is_alpha (only letters?)	is_punct (punctuation?)	like_num (looks like a number?)
`costs`	**True**	False	False
`$`	False	False	False
`10.50`	False	False	**True**
`U.S.`	False	False	False
`km`	**True**	False	False
`!`	False	**True**	False

These attributes help us filter text. For example, we can remove all punctuation or collect all numbers.

Other useful attributes include the word type (`pos_`), the base form (`lemma_`), and whether it is a stop word (`is_stop`).

02

Parts of Speech

Giving every word a job title.

PARTS OF SPEECH

Every word has a job

Parts-of-Speech (POS) tagging labels each token with its grammar job, like noun, verb, or adjective.

The	quick	brown	fox	jumps	over	the	lazy	dog	.
DET	ADJ	ADJ	NOUN	VERB	ADP	DET	ADJ	NOUN	PUNCT

Why does this matter?

The same word can have different jobs. The tag tells the computer which meaning is being used.

Example with “book”

“I read a **book**.” → NOUN

“Please **book** a flight.” → VERB

PARTS OF SPEECH

The common POS tags

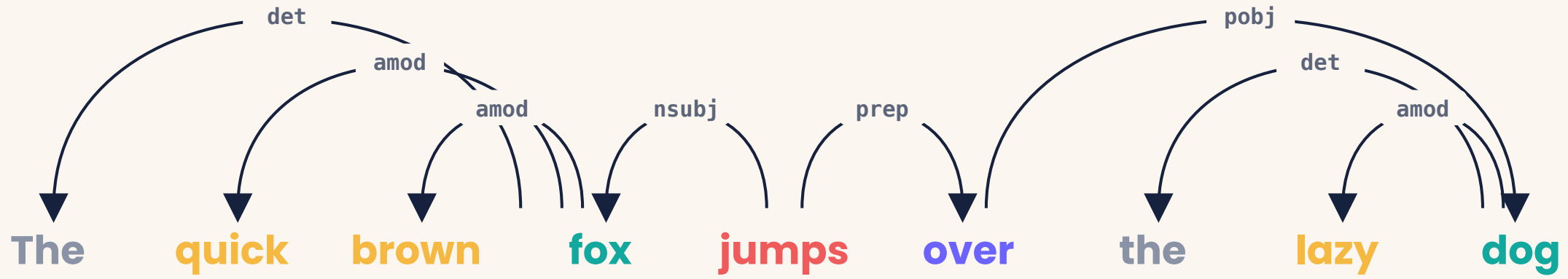
spaCy uses a standard set of short tags. You do not need to memorize them. Just know where to look.

`ADJ`	**Adjective** *big, happy, round*	`ADP`	**Adposition** *in, at, of, under*	`ADV`	**Adverb** *quickly, silently*
`AUX`	**Auxiliary verb** *is, has, do*	`CCONJ`	**Coordinating conjunction** *and, but, or*	`DET`	**Determiner** *a, the, some*
`INTJ`	**Interjection** *wow, ouch*	`NOUN`	**Noun** *dog, city, happiness*	`NUM`	**Numeral** *one, two, 3.14*
`PART`	**Particle** *to (in "to run")*	`PRON`	**Pronoun** *he, she, it, they*	`PROPN`	**Proper noun** *John, Paris, Tesla*
`PUNCT`	**Punctuation** *, . ! ?*	`SCONJ`	**Subordinating conjunction** *if, while, that*	`SYM`	**Symbol** *$, %, @*
`VERB`	**Verb** *run, think, eat*	`X`	**Other** *unknown or foreign words*		

SIMULATED DEMO

Seeing how words relate to each other

spaCy can draw arrows that show how words connect. This is called a **dependency** view.



nsubj = who does the action **amod** = a describing word **det** = a, an, the **prep / pobj** = a place or direction

Reading the arrows: “quick” and “brown” describe the fox, and the fox is the one who jumps.

03

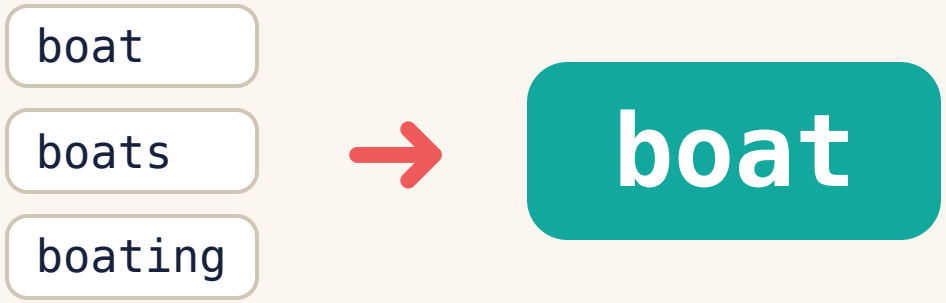
Stemming

Chopping words down to their root.

STEMMING

What is stemming?

Stemming removes the ending of a word to find its **stem**, or root. It uses simple cutting rules.



Why this helps

When you search for “boat,” you probably also want results that say “boats” or “boating.”

Think of a gardener

A gardener trims the extra leaves so only the main stem is left. A stemmer trims the extra letters.

STEMMING

Why is stemming useful?

1

Text normalization

Different forms like “runs” and “running” become one form.

2

Better search

Searching “run” can also find “runs” and “running.”

3

Smaller vocabulary

Fewer unique words means less work for the computer.

4

Better ML models

Models focus on meaning instead of grammar differences.

5

Helps complex languages

Languages with many word endings, like Finnish and Turkish, become easier to analyze.

6

Better grouping

Texts with similar words are grouped together more easily.

STEMMING

Before and after stemming

Without stemming

`connect` `connected` `connecting` `connection` `connections`

5 different words

The computer thinks these are five unrelated words.

With stemming

`connect` `connect` `connect` `connect` `connect`

1 word

Now the computer knows they all share the same idea.

Stemming makes text processing simpler, faster, and more consistent.

The Porter Stemmer: Phase 1

The **Porter Stemmer** (1980) is one of the most famous stemmers. It cuts words in **five phases**, and each phase has its own rules.

Rule	Word	Stem
SSES → SS	caresses	caress
IES → I	ponies	poni
SS → SS	caress	caress
S → (remove)	cats	cat

Phase 1 uses simple rules that remove plural endings.

Longest match wins. Only one rule is used, based on the longest matching ending. That is why “caresses” becomes “caress” and not “cares.”

Notice “poni.” A stem does **not** have to be a real word.

The Porter Stemmer: smarter phases

Later phases are smarter. They check if the word is **long enough** before cutting.

Rule	Word	Result of this step
(m>0) ATIONAL → ATE	relational	**relate**
	national	national (no change)
(m>0) EED → EE	agreed	**agree**
	feed	feed (no change)

What is (m>0)? It means “only cut if enough of the word is left.” After removing “ational,” **national** would leave only “n,” which is too short.

In **feed**, only “f” would be left before “eed,” so the rule does not apply.

Remember that later phases may cut even more. The final stem of “relational” is “relat.”

SIMULATED DEMO

Snowball: the improved stemmer

SIMULATED DEMO

```
from nltk.stem import PorterStemmer, SnowballStemmer

p = PorterStemmer()
s = SnowballStemmer("english")

for w in ["running", "easily", "fairly",
"generously"]:
    print(w, p.stem(w), s.stem(w))
```

Word	Porter	Snowball
running	run	run
easily	easili	easili
fairly	fairli	**fair**
generously	gener	**generous**

The **Snowball Stemmer** is also called the “Porter2” or “English Stemmer.”

It is a small upgrade to Porter. It is slightly more accurate and faster.

Look at “generously.” Porter cuts it to “gener,” while Snowball keeps the clearer stem “generous.”

STEMMING

The problems with stemming

Over-stemming

The stemmer cuts too much and joins unrelated words.

```
universe → univers
```

```
university → univers
```

Fake words

The stem is often not a real word, which makes it hard for people to read.

```
happiness → happi
```

```
studies → studi
```

Lost meaning

Stemming ignores context, so meaning can be lost completely.

```
organization → organ
```

```
organize → organ
```

Is an “organization” a body part? Of course not! We need a smarter method.

04

Lemmatization

Finding the true dictionary form of a word.

LEMMATIZATION

What is lemmatization?

Lemmatization changes a word into its dictionary form, called the **lemma**. It looks at the whole vocabulary and the word's part of speech.

Stemming is like scissors ✂️

It cuts the end of a word by following rules, even if the result is not a real word.

`studies → studi`

Lemmatization is like a dictionary 📖

It checks what the word really is and returns a real, meaningful word.

`studies → study`

Fun fact: spaCy does not even include a stemmer. It uses lemmatization instead because it is more accurate.

Stemming vs. lemmatization

Feature	Stemming	Lemmatization
How it works	Cuts the word using simple rules.	Uses a dictionary and grammar to find the base form.
Uses context?	No. It ignores the part of speech.	Yes. It checks the part of speech.
Output	May create words that do not exist.	Always gives a real dictionary word.
Speed	Very fast.	A little slower.
Example	`studies → studi`	`studies → study`

Why lemmatization is better: three examples

1. Verb forms

Word	Stem	Lemma
running	run	**run**
ran	ran	**run**
runs	run	**run**

The lemma knows that “ran” is a past form of “run.”

2. Irregular nouns

Word	Stem	Lemma
geese	gees	**goose**
feet	feet	**foot**

The dictionary knows special plurals that rules cannot handle.

3. No over-cutting

Word	Stem	Lemma
organization	organ	**organization**
organize	organ	**organize**

The lemma keeps the correct meaning of each word.

LEMMATIZATION

Context changes the answer

The same word can have a different lemma depending on its job in the sentence.

*"I am **meeting** my friends later."*

VERB The lemma is **meet**, because it is an action.

*"The **meeting** was very long."*

NOUN The lemma stays **meeting**, because it is an event.

A stemmer gives "meet" both times. A lemmatizer understands the difference.

SIMULATED DEMO

Lemmatization with spaCy

SIMULATED DEMO

```
text = ("I am a runner running in a race "
        "because I love to run "
        "since I ran today")
doc = nlp(text)
for token in doc:
    print(token.text, token.pos_, token.lemma_)
```

I PRON I	**am** AUX be	**a** DET a	**runner** NOUN runner	**running** VERB run	**in** ADP in
a DET a	**race** NOUN race	**because** SCONJ because	**I** PRON I	**love** VERB love	**to** PART to
run VERB run	**since** SCONJ since	**I** PRON I	**ran** VERB run	**today** NOUN today	

am → be. spaCy knows that “am” is a form of the verb “be.”

running → run and **ran → run.** All forms of the action become one lemma.

runner → runner. It stays the same because a runner is a person, which is a noun and not an action.

Advantages of lemmatization

🎯

1. Accuracy

It gives correct dictionary words and uses the part of speech.

🧠

2. Context awareness

It handles irregular words like “geese” and words that change meaning, like “meeting.”

📖

3. Readability

Its results are easy to read, which helps in tasks like summarizing and translating.

So is stemming useless?

No. Stemming is still useful when you need **speed** on a huge amount of text, like a simple search engine. Choose the tool that fits the task.

05

Stop Words

Filtering out very common words.

STOP WORDS

What are stop words?

Stop words are very common words, like *the, is, in,* and *and*. They appear often but add little meaning, so we often remove them.

~~The~~ quick brown fox jumps ~~over~~ ~~the~~ lazy dog .

↓ remove stop words

quick brown fox jumps lazy dog

The main meaning stays the same, but there is less text to process. Common stop words include: *is, are, the, a, an, of, in, on, and, it, to.*

SIMULATED DEMO

Working with stop words in spaCy

SIMULATED DEMO

```
# How many stop words does spaCy know?
print(len(nlp.Defaults.stop_words))

# Is "the" a stop word?
print(nlp.vocab["the"].is_stop)

# Add our own stop word
nlp.Defaults.stop_words.add("btw")
nlp.vocab["btw"].is_stop = True
```

OUTPUT

```
326
True
```

spaCy comes with a built-in list of **326** English stop words.

We can check any word to see if it is on the list.

We can also **add** or **remove** words. For example, you could add chat slang like “btw” if it does not matter for your project.

STOP WORDS

Careful! Stop words can matter

The word **“not”** is on spaCy’s stop word list. See what happens to this movie review.

> *“This movie is not good at all.”*

↓ remove all stop words

`movie` `good`

The problem

The review was negative, but now it looks positive. The meaning flipped completely.

The lesson

Always think about your task. For sentiment analysis, keep words like “not,” “no,” and “never.”

06

Vocabulary and Matching

Finding phrases with patterns we design.

What is rule-based matching?

Rule-based matching finds phrases in text that fit a **pattern** we define ourselves. Think of it as a smarter version of Ctrl+F.

Ctrl+F

It only finds the **exact** letters you type. Searching “solar power” misses “Solar-power” and “solarpower.”

spaCy Matcher

It describes tokens, not just letters. A pattern can check the lowercase text, punctuation, or even the part of speech.

A pattern is a list of rules, one rule for each token we want to match.

SIMULATED DEMO

Matching different spellings

SIMULATED DEMO

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)

p1 = [{"LOWER": "solarpower"}]
p2 = [{"LOWER": "solar"}, {"IS_PUNCT": True},
{"LOWER": "power"}]
p3 = [{"LOWER": "solar"}, {"LOWER": "power"}]
matcher.add("SolarPower", [p1, p2, p3])
```

OUTPUT

```
Solar Power
Solar-power
solarpower
```

Text: “The Solar Power industry keeps growing. Solar-power panels are cheaper now, and solarpower is popular.”

p1 finds one joined word. **p2** finds two words with punctuation between them. **p3** finds two separate words.

Using **LOWER** means capital letters do not matter.

SIMULATED DEMO

Matching with parts of speech

SIMULATED DEMO

```
pattern = [{"POS": "ADJ"}, {"POS": "NOUN"}]
matcher.add("AdjNoun", [pattern])

text = ("The friendly teacher gave us "
        "a difficult quiz about smart robots.")
doc = nlp(text)
```

OUTPUT

```
friendly teacher
difficult quiz
smart robots
```

This pattern means: “find an **adjective** followed by a **noun**.”

We never typed the actual words. spaCy used the part-of-speech tags to find them.

Real use: A store can find how customers describe products, like “cheap charger” or “broken screen.”

SIMULATED DEMO

Matching a list of words

SIMULATED DEMO

```
from spacy.matcher import PhraseMatcher
matcher = PhraseMatcher(nlp.vocab, attr="LOWER")

foods = ["adobo", "sinigang", "halo-halo"]
matcher.add("Food", [nlp.make_doc(f) for f in
foods])

doc = nlp("We ate Adobo for lunch and "
          "Halo-Halo for dessert.")
```

OUTPUT

```
Adobo
Halo-Halo
```

A **PhraseMatcher** is best when you already have a **list of terms** to find.

Instead of writing a pattern for each item, you just give it the list.

Real use: finding product names, school names, or places in thousands of messages.

PUTTING IT ALL TOGETHER

One review, the full pipeline

“The movies weren’t boring at all! I loved the actors.”

1. Tokens

`The` `movies` `were` `n't` `boring` `at` `all` `!` `I` `loved` `the` `actors` `.`

2. POS tags

`DET` `NOUN` `AUX` `PART` `ADJ` `ADV` `ADV` `PUNCT` `PRON` `VERB` `DET` `NOUN` `PUNCT`

3. Lemmas

`the` `movie` `be` `not` `boring` `at` `all` `!` `I` `love` `the` `actor` `.`

4. Keep key words

`movie` `not` `boring` `love` `actor`

Result: “not boring” + “love” → a **positive** review. We kept “not” on purpose!

MOTIVATION

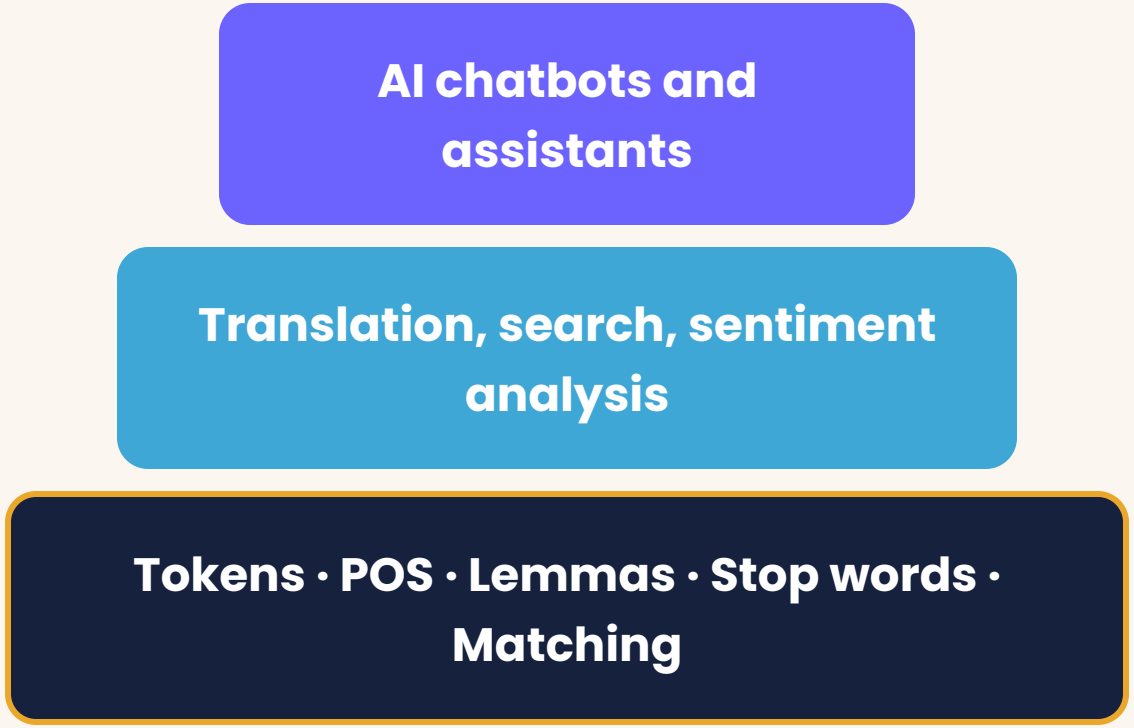
From today’s lesson to modern AI

The steps you learned today are not old history. They are the foundation of modern AI.

Today’s AI chatbots also begin by **breaking your message into tokens**.

Search engines still use ideas like **stemming**, **lemmas**, and **matching** to find what you want.

NLP changes very fast, with new progress every month. But every new method is built on these **basic ideas**.



You learned the foundation today.

SUMMARY

Key takeaways

1. NLP helps computers read, understand, and use human language.
2. Tokenization breaks text into small pieces called tokens.
3. POS tagging labels the job of each word, like noun or verb.
4. Stemming cuts words to a root. It is fast but can make mistakes.
5. Lemmatization finds the real dictionary form using context.
6. Stop words are common words. Remove them carefully, and keep words like “not” when meaning matters.
7. Matching finds phrases using patterns we design.

CHECK YOUR UNDERSTANDING

Quick quiz

Q1

What is a token?

A. A computer password
B. A small piece of text, like a word or punctuation mark
C. A type of stop word

Q2

What does stemming turn “boating” into?

A. boat
B. boater
C. ship

Q3

Which method turns “geese” into “goose”?

A. Tokenization
B. Stemming
C. Lemmatization

Q4

Why should we keep “not” when analyzing reviews?

A. It is a noun
B. Removing it can flip the meaning
C. It is too short

Q5

The pattern ADJ + NOUN would find which phrase?

A. smart robots
B. runs fast
C. in the house

CHECK YOUR UNDERSTANDING

Quiz answers

Q1 **Answer: B**
A token is a small piece of the original text.

Q2 **Answer: A**
The stemmer cuts off “-ing” and leaves “boat.”

Q3 **Answer: C**
Lemmatization uses a dictionary to handle irregular words.

Q4 **Answer: B**
“Not good” becomes “good” without it.

Q5 **Answer: A**
“smart” is an adjective and “robots” is a noun.

ACTIVITY

Try it yourself: be the computer

Work with a partner. Use this sentence and act like an NLP pipeline by hand.

> *“The students weren’t studying for the tests, but they loved the games!”*

Task 1: Tokenize

Split the sentence into tokens. How many tokens did you get? Watch out for “weren’t” and punctuation.

Task 2: Tag

Label each word as a noun, verb, adjective, or other.

Task 3: Lemmatize

Write the dictionary form of “students,” “studying,” “tests,” and “loved.”

Task 4: Think

Circle the stop words. Which ones should you keep so the meaning does not change?

NATURAL LANGUAGE PROCESSING

Thank you for listening!

Now you know how computers start to understand our words.
What questions do you have?

`Keep` `learning` `,` `keep` `building` `!`